Assignment 6.2: Evaluating Ethical Risks and Responsible AI Practices in DevOps

Date: September 19, 2026

Course: WEB430

Professor: Richard Krasso

Student: Jonathan Cantu

Part I: Evaluating Ethical Risks and Responsible AI Practices at ClearWave Health

ClearWave Health is already seeing the productivity benefits of AI-assisted development, but its current practices expose the organization to unnecessary risk. In my experience working on large, regulated systems, automation is most valuable when it creates consistency and reduces repetitive work without reducing accountability. The central issue at ClearWave is not that AI is being used, but that its use is inconsistent, insufficiently documented, and sometimes trusted beyond what the evidence supports. In a healthcare-related system, that is especially concerning because software decisions can influence scheduling, billing, system availability, and ultimately the people depending on those services.

1. Identify Ethical Risks

One of ClearWave's largest risks is inconsistent review of AI-generated code. AI can produce code that appears correct while still containing subtle defects, security weaknesses, or assumptions that do not align with the system's business rules. Krasso (2026) notes that incorrect or misleading AI output can scale quickly when inserted into automated DevOps workflows. The danger is not necessarily an obviously bad answer; it is an answer that looks reasonable enough that nobody questions it.

The lack of documentation about where AI is being used creates an additional transparency and accountability problem. If a defect is later discovered, the team should be able to determine whether code, test cases, monitoring recommendations, or incident actions were AI-assisted and who reviewed them. Microsoft identifies transparency and accountability as core principles of responsible AI, along with reliability, safety, privacy, and security.

Automated test generation also presents risk when critical edge cases are missed. AI-generated tests may improve coverage, but they can reflect gaps in training data or assumptions about normal behavior. Krasso (2026) specifically warns that bias can appear in test generation when certain cases are underrepresented.

Finally, inaccurate monitoring alerts and over-reliance on AI during incidents may lead teams toward the wrong diagnosis or remediation. During an incident, speed matters, but confidence and context matter just as much. An AI-generated recommendation should be treated as evidence to evaluate, not as an instruction to follow automatically.

2. Evaluate AI-Assisted Practices

ClearWave is using AI effectively in several areas. Generating initial test cases, analyzing monitoring signals, assisting developers with code, and supporting incident analysis can all reduce cognitive load and shorten feedback loops. These are appropriate uses because AI is helping the team process information or produce a first draft faster.

The weakness is in oversight. AI-generated code is not reviewed consistently, generated test cases are apparently accepted despite coverage gaps, monitoring alerts are sometimes inaccurate, and incident teams appear to be placing too much confidence in AI recommendations. The organization also lacks an audit trail showing when and where AI contributed to the workflow.

From my perspective, that moves AI from an assistant toward an undocumented decision-maker. ClearWave should instead use AI to surface information, identify patterns, and recommend actions while maintaining clear human ownership of consequential decisions.

3. Recommend Responsible AI Guidelines

I would recommend several practical guidelines.

First, all AI-generated or substantially AI-modified code should go through the same pull request, automated testing, static analysis, and human review process as code written entirely by a developer. AI should never become a shortcut around established quality gates.

Second, ClearWave should document AI usage. Pull requests could include a simple indication that AI assisted with code, testing, or documentation and identify what was reviewed by the developer. This does not need to become excessive bureaucracy; the goal is traceability.

Third, AI-generated tests should be treated as a starting point rather than proof of adequate coverage. Developers should validate expected behavior and specifically review edge cases, failure scenarios, and business rules that an AI model may not understand.

Fourth, monitoring and incident-response tools should explain why they produced a recommendation whenever possible. Krasso (2026) emphasizes that explainability helps teams determine whether an AI recommendation is reasonable and whether it is based on incorrect assumptions.

Finally, ClearWave should establish risk-based human approval. AI might automatically summarize logs or identify anomalies, but actions affecting production systems, sensitive data, security, architecture, or significant customer impact should require human authorization. Microsoft similarly states that people remain accountable for AI systems and that highly autonomous systems should maintain meaningful human control.

4. Connect to the Three Ways

Responsible AI practices support all Three Ways.

Flow improves when AI handles repetitive work such as initial code analysis, test brainstorming, log correlation, and documentation while clear policies prevent teams from repeatedly stopping to determine whether an AI action is acceptable.

Feedback improves when AI output is validated through automated tests, peer review, monitoring, and explainable alerts. The important point is that faster feedback must also be trustworthy feedback.

Continuous Learning improves when ClearWave tracks where AI recommendations were correct, where they failed, what edge cases were missed, and how human reviewers responded. Instead of treating AI mistakes as isolated incidents, the team can use them to improve prompts, policies, testing practices, and workflow controls over time.

The goal should not be to limit AI simply because it introduces risk. It should be to use AI where it improves the team's effectiveness while creating enough transparency, validation, and human oversight to keep that automation trustworthy. AI can accelerate engineering work, but ClearWave's developers and operational teams must remain accountable for the systems they build and operate.

References:

Krasso, Richard (2026). "WEB 430 Weekly Reading: Week 6". Bellevue University.

Microsoft. (2026). “What is Responsible AI?” Microsoft Learn. https://learn.microsoft.com/en-us/shows/responsible-ai/

Part II: AI-Assisted Code Review and Ethical Evaluation

For this part of the assignment, you will work with a code snippet and use an AI-assisted tool to:

- Generate code improvements
- Generate documentation
- Generate test cases
- Evaluate the ethical implications of the AI's output

This gives you practical experience reviewing AI-generated output with a critical mindset.

Code Snippet (JavaScript):

```
function assignPriority(user) {
  if (user.role === "admin") {
    return "high";
  }

  if (user.role === "manager") {
    return "medium";
  }

  return "low";
}
```

Steps:

1. Copy the code snippet into an AI-assisted tool.
2. Ask the tool to suggest improvements.
3. Ask the tool to generate documentation.
4. Ask the tool to generate test cases.
5. Review the output and evaluate potential ethical concerns.
6. Reflect on what you notice.

What to Submit for the Hands-on Portion:

1. AI-generated improvements
2. AI-generated documentation
3. AI-generated test cases
4. A short reflection (8-10 sentences) addressing:
5. What the AI did well
6. What it missed
7. Any ethical concerns you identified
8. How this connects to responsible AI practices

Response

For Part II, I used Deepseek as the AI Tool. Here's the Deepseek prompt and code:

Given the following code snippet:

```function assignPriority(user) {
if (user.role === "admin") {
return "high";
}

if (user.role === "manager") {
return "medium";
}

return "low";
}
```

1. Suggest improvements.
2. Generate documentation.
3. Generate test cases.

Deepseek Response

🚀 Code Analysis & Improvements

1️⃣ Suggested Improvements

🎯 Current Issues & Improvements

Issue	Improvement	Why
Hardcoded strings	Use constants/enums	Prevents typos, enables autocomplete
No default role handling	Explicit default case	Clearer intent when role is undefined/missing
No type checking	TypeScript/JSDoc	Better developer experience, catches errors early
No input validation	Guard clause	Prevents runtime errors with invalid inputs
Magic values	Extract to configuration	Easier maintenance and updates

💻 Improved Code

```
// Option 1: Using constants (Vanilla JS)
const PRIORITY_LEVELS = {
  HIGH: 'high',
  MEDIUM: 'medium',
```

```
  LOW: 'low'
};

const USER_ROLES = {
  ADMIN: 'admin',
  MANAGER: 'manager'
};

function assignPriority(user) {
  // Input validation
  if (!user || typeof user !== 'object') {
    throw new TypeError('Expected user object');
  }

  // Role-based priority assignment
  switch (user.role) {
    case USER_ROLES.ADMIN:
      return PRIORITY_LEVELS.HIGH;
    case USER_ROLES.MANAGER:
      return PRIORITY_LEVELS.MEDIUM;
    default:
      return PRIORITY_LEVELS.LOW;
  }
}

// Option 2: Using TypeScript (Recommended)
type UserRole = 'admin' | 'manager' | 'guest' | string;
type PriorityLevel = 'high' | 'medium' | 'low';
```

```
interface User {
  role: UserRole;
  // other properties...
}

const assignPriority = (user: User): PriorityLevel => {
  if (!user) throw new Error('User is required');

  switch (user.role) {
    case 'admin': return 'high';
    case 'manager': return 'medium';
    default: return 'low';
  }
};
```

🔧 Additional Enhancements

```
// Configuration-driven approach (most flexible)
const ROLE_PRIORITY_MAP = {
  admin: 'high',
  manager: 'medium',
  guest: 'low',
  viewer: 'low'
  // Add new roles here without modifying function
};

function assignPriority(user) {
  if (!user || typeof user !== 'object') {
    throw new TypeError('User object is required');
  }
```

```
  return ROLE_PRIORITY_MAP[user.role] || 'low';
}
```

---

2 Documentation 📖

JSDoc Format

```
/**
 * Assigns a priority level to a user based on their role
 *
 * @param {Object} user - The user object containing role information
 * @param {string} user.role - The user's role (e.g., 'admin', 'manager')
 *
 * @returns {string} Priority level: 'high', 'medium', or 'low'
 *
 * @throws {TypeError} If user is not an object or is null/undefined
 *
 * @example
 * // Returns 'high'
 * assignPriority({ role: 'admin' });
 *
 * @example
 * // Returns 'low'
 * assignPriority({ role: 'guest' });
 *
 * @example
 * // Returns 'low' (default for missing/invalid roles)
 * assignPriority({ role: 'unknown' });
 */
function assignPriority(user) {
  // ... implementation
```

```
}
```

README Documentation

```
## Priority Assignment Function


### Purpose

Assigns priority levels to users based on their system role.


### Priority Mapping

| Role | Priority ||------|----------||`admin`|`high`||`manager`|`medium`|| Any other role |`low`|

### Usage

```javascript

const user = { role: 'admin', name: 'Alice' };

const priority = assignPriority(user); // Returns 'high'
```

Error Handling

- Throws `TypeError` if `user` is not an object
- Returns `'low'` for any undefined or unrecognized role

```
---


## 3️⃣ Test Cases 🧪


### **Test Suite Structure**


```javascript

// assignPriority.test.js

describe('assignPriority', () => {


  // Positive test cases

  describe('valid user roles', () => {

    test('returns "high" for admin role', () => {
```

```
    expect(assignPriority({ role: 'admin' })).toBe('high');
  });

  test('returns "medium" for manager role', () => {
    expect(assignPriority({ role: 'manager' })).toBe('medium');
  });

  test('returns "low" for guest role', () => {
    expect(assignPriority({ role: 'guest' })).toBe('low');
  });

  test('returns "low" for undefined role', () => {
    expect(assignPriority({ role: undefined })).toBe('low');
  });

  test('returns "low" for empty string role', () => {
    expect(assignPriority({ role: '' })).toBe('low');
  });
});

// Edge cases
describe('edge cases', () => {
  test('handles role with additional user properties', () => {
    const user = {
      role: 'admin',
      name: 'Alice',
      email: 'alice@example.com'
    };
    expect(assignPriority(user)).toBe('high');
  });
```

```
  test('is case-sensitive (does not match "Admin")', () => {
    expect(assignPriority({ role: 'Admin' })).toBe('low');
  });

  test('handles numeric role (coerced to string)', () => {
    expect(assignPriority({ role: 123 })).toBe('low');
  });
});

// Error cases
describe('error handling', () => {
  test('throws TypeError for null user', () => {
    expect(() => assignPriority(null)).toThrow(TypeError);
  });

  test('throws TypeError for undefined user', () => {
    expect(() => assignPriority(undefined)).toThrow(TypeError);
  });

  test('throws TypeError for non-object user', () => {
    expect(() => assignPriority('not an object')).toThrow(TypeError);
    expect(() => assignPriority(42)).toThrow(TypeError);
    expect(() => assignPriority([])).toThrow(TypeError);
  });

  test('handles missing role property gracefully', () => {
    expect(assignPriority({ name: 'Bob' })).toBe('low');
  });
});
```

```
 // Future-proofing
 describe('extensibility', () => {
  test('can handle new roles without code changes (with config approach)', () => {
    // If using configuration object approach
    const testConfig = { admin: 'high', manager: 'medium', vip: 'high' };
    // Mock implementation for testing
    const assignPriorityWithConfig = (user) =>
      testConfig[user.role] || 'low';

    expect(assignPriorityWithConfig({ role: 'vip' })).toBe('high');
  });
 });
});
```

Test Coverage Summary

Test Category	Coverage
✅ Valid inputs	All defined roles, additional properties
⚠️ Edge cases	Case sensitivity, numeric roles, empty strings
🚨 Error handling	Invalid inputs, missing properties
🔮 Future-proofing	New roles, configuration changes

Quick Test Matrix

```
// Quick reference test matrix
const testMatrix = [
 // [input, expectedOutput, description]
 [{ role: 'admin' }, 'high', 'Admin gets high priority'],
 [{ role: 'manager' }, 'medium', 'Manager gets medium priority'],
 [{ role: 'guest' }, 'low', 'Guest gets low priority'],
 [{ role: 'viewer' }, 'low', 'Viewer gets low priority'],
 [{ role: '' }, 'low', 'Empty role gets low priority'],
 [{ role: undefined }, 'low', 'Undefined role gets low priority'],
```

```
  [{}, 'low', 'Missing role property gets low priority'],
  // Error cases
  [null, 'throws', 'Null user throws error'],
  [undefined, 'throws', 'Undefined user throws error'],
  ['string', 'throws', 'String input throws error'],
];
```

🏁 Summary Recommendations

Priority 1 (Must Do)

1. **Add input validation** – Prevent runtime errors
2. **Document the function** – JSDoc for developer clarity
3. **Write basic tests** – Cover admin/manager/other roles

Priority 2 (Should Do)

1. **Extract to configuration** – Easier to add new roles
2. **Add TypeScript types** – Better type safety
3. **Add edge case tests** – Handle undefined/null inputs

Priority 3 (Nice to Have)

1. **Support role inheritance** – If users have multiple roles
2. **Add logging** – For debugging priority assignments
3. **Make priority configurable** – Allow custom priority levels

Best improvement for maintainability: Switch to a configuration object approach – it separates business logic from data and makes the system more adaptable to future role changes! 🎯

AI-Assisted Code Review and Ethical Evaluation

AI-Generated Improvements

DeepSeek identified several technical weaknesses in the original `assignPriority()` function. It recommended adding input validation so invalid or missing user objects do not cause runtime errors, replacing hardcoded strings with constants or configuration values, and documenting the expected user roles and priority values.

DeepSeek also suggested a configuration-driven approach:

Javascript:

```
const ROLE_PRIORITY_MAP = {
  admin: 'high',
  manager: 'medium',
  guest: 'low',
  viewer: 'low'
};

function assignPriority(user) {
  if (!user || typeof user !== 'object') {
    throw new TypeError('User object is required');
  }

  return ROLE_PRIORITY_MAP[user.role] || 'low';
}
```

This approach separates the role-to-priority mapping from the function logic, which could make future changes easier to maintain. DeepSeek also suggested that TypeScript could provide stronger type checking if the project uses it.

AI-Generated Documentation

DeepSeek generated JSDoc documentation describing the function's purpose, input, return value, error behavior, and example usage. Its documentation explained that the function assigns a priority level based on a user's role and returns `high`, `medium`, or `low`.

It also generated README-style documentation containing a role-to-priority mapping:

- `admin` → `high`
- `manager` → `medium`
- all other roles → `low`

DeepSeek also documented its proposed validation behavior, including throwing a `TypeError` when the supplied user is not a valid object.

AI-Generated Test Cases

DeepSeek generated tests covering normal inputs, edge cases, and invalid inputs. Examples included:

Javascript:

```
test('returns "high" for admin role', () => {
  expect(assignPriority({ role: 'admin' })).toBe('high');
});

test('returns "medium" for manager role', () => {
  expect(assignPriority({ role: 'manager' })).toBe('medium');
});

test('returns "low" for guest role', () => {
  expect(assignPriority({ role: 'guest' })).toBe('low');
});

test('is case-sensitive', () => {
  expect(assignPriority({ role: 'Admin' })).toBe('low');
});

test('throws TypeError for null user', () => {
  expect(() => assignPriority(null)).toThrow(TypeError);
});
```

It also suggested tests for missing role values, empty strings, numeric roles, additional user properties, and future configuration changes.

Reflection

I thought DeepSeek did a good job identifying technical weaknesses such as missing input validation, hardcoded values, and limited test coverage. It also generated useful documentation and several edge-case tests that would improve maintainability and make the behavior of the function clearer. However, the most important thing it missed was questioning whether the underlying business rule itself is appropriate. The code assumes that an administrator should receive high priority, a manager medium priority, and everyone else low priority, but DeepSeek largely accepted that rule as valid without asking what "priority" actually represents. In a healthcare-related application, that distinction could become ethically significant if priority affects access to services, incident response, billing treatment, or patient-facing workflows. Krasso (2026) notes that bias can hide in assumptions and patterns that appear reasonable on the surface, which is exactly the concern I see here. DeepSeek also suggested adding roles such as `guest`, `viewer`, and `vip`, but those recommendations introduce additional business assumptions that were never provided in the original requirements. The generated tests validate that the code follows the stated role hierarchy, but they do not validate whether that hierarchy is fair, justified, or compliant with the application's actual requirements. Responsible AI use therefore requires developers to review not only whether generated code is technically correct, but also whether the assumptions behind it are appropriate. In this example, I would want a human reviewer or product owner to confirm the meaning of priority and the legitimate criteria for assigning it before accepting any AI-generated implementation.

Reference:

Krasso, Richard (2026). "WEB 430 Weekly Reading: Week 6". Bellevue University.